

08 | List：如何实现高性能的无限列表？

2022-4-13 蒋宏伟



你好，我是蒋宏伟。今天我们学习的重点是列表组件 `RecyclerView`。

如果你熟悉 `React Native`，那你可能会问了：“`React Native` 中的列表组件不是 `FlatList` 吗？”

没错。`React Native` 官方提供的列表组件确实是 `FlatList`，但是我推荐你优先使用开源社区提供的列表组件 `RecyclerView`。因为，开源社区提供的 `RecyclerView` 性能更好。

对于列表组件来说，我们最应该关心的就是性能。这里我给你分享下我的个人经历。2016~2018 年，我参与了一个用 `React Native` 搭建的信息流项目。信息流这种无限列表页是非常常见的业务场景，比如你使用的京东首页、抖音视频、微信朋友圈都属于信息流页面。你看完一页，还有下一页，看完下一页还有下下页，无穷无尽。这时就要用到我们马上要探讨的列表组件了，而且必须是高性能的列表组件，不能翻着翻着就卡起来了。

2016 年，没有 `RecyclerView`，也没有 `FlatList`，我们用的是第一版的 `ListView` 组件。`ListView` 组件性能很差，没有内存回收机制，翻一页内存就涨一点，再翻一页内存又再涨一点。前 5 页滚动非常流畅，第 10 页开始就感觉到卡顿，到 50 页的时候，基本就滑不动

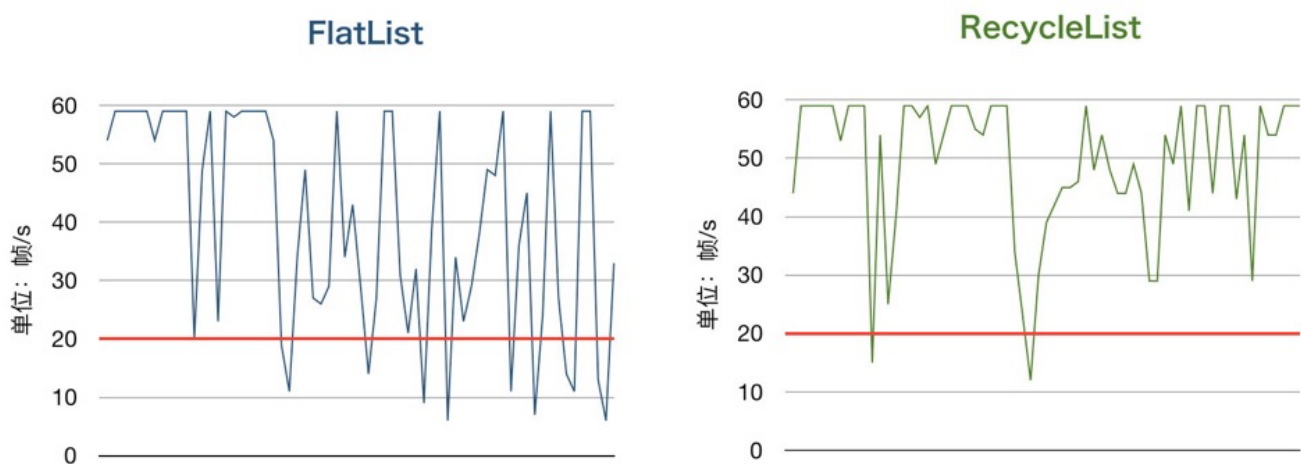
了。卡顿的原因就是无限列表太吃内存了。如果手机的可使用内存不够了，卡顿就会发生。这也是 React Native 刚出来时被吐槽得最多的地方。

2017 年，官方的第二代列表组件 FlatList 出来后，第一代列表组件 ListView 就被废弃了，这时候无限列表性能变得好一些了。虽然 FlatList 在 iOS 端表现很好，但在 Android 低端机还是能感觉到卡顿。

2018 年，随着业务越来越复杂，FlatList 的性能表现变得更加糟糕了。经过调研，我们找到了性能更好的列表组件 RecyclerView。通常评判列表卡顿的指标是 UI 线程的帧率和 JavaScript 线程的帧率。

但业内有人实验过，在已经渲染完成的页面中，通过死循环把 JavaScript 线程卡死，页面依旧能够滚动。这是因为滚动本身是在 UI 线程进行的，和 JavaScript 线程无关。但当用户下滑，需要渲染新的列表项时，就需要 JavaScript 线程参与进来了。如果这时候 JavaScript 掉帧了，新的列表项就渲染不出来，即便能滚动，用户看到也是空白项，一样影响用户体验。

因此，我们当时是把 JavaScript 帧率作为客观指标，再加上团队同学主观体验，进行综合评估。采集 JavaScript 帧率用的手机是 OPPO R9，现在看来是妥妥的低端机了，结果显示，FlatList JavaScript 帧率小于 20 帧的占比有 16%，而 RecyclerView 占比只有 3%。主观体验上，团队同学拿自己的手机进行测试，使用暴力滑动的测试方法，测评了 20 来款机型。在低端机上 FlatList 多被标记为一般卡，而 RecyclerView 大多标记是流畅，只有少量的轻微卡顿。



即使现在新架构马上就要出来了，在这个时间点上，我最推荐你用的还是 RecyclerView。因为从原理上 RecyclerView 比 FlatList 强上不少。

作为一个开发者，你总有需要手动优化的时候，不是所有场景都有现成的组件，都有自动化的解决方案。如果你现在没有遇到，兴许只是因为开发年头太少了，你可以问问你身边那些开发年头多的同学，他们在这方面应该是很深的体会。学习 FlatList、RecyclerView 的优化原理，对自己的动手优化是非常有帮助的。当你以后遇到列表性能问题时，你可以有现成的优化思路借鉴，不会毫无头绪。

那么，为什么开源社区的 RecyclerView 比官方的 FlatList 性能更好？FlatList、RecyclerView 的优化原理是什么？FlatList 和 RecyclerView 的底层实现都是滚动组件 ScrollView，所以我们先从 ScrollView 聊起。

ScrollView：渲染所有内容的滚动组件

ScrollView 是一个支持横向或竖向的滚动组件，几乎所有页面都会用到。

ScrollView 组件类似于 Web 中的 `<html />` 或 `<body />` 标签，浏览器中的页面之所以能上下滚动，就是因为 html 或 body 标签默认有一个 `overflow-y: scroll` 的属性，如果你把标签的属性设置为 `overflow-y: hidden`，页面就不能滚动了。

React Native 的 ScrollView 组件在 Android 的底层实现用的是 ScrollView 和 HorizontalScrollView，在 iOS 的底层实现用的是 UIScrollView。

所谓的滚动，解决的是在有限高度的屏幕内浏览无限高度的内容的问题。有限高度的容器是 ScrollView，无限高度，或者说高度不确定的内容是 ScrollView 的 children。

使用 ScrollView 组件时，我们通常并不直接给 ScrollView 设置固定高度或宽度，而是给其父组件设置固定高度或宽度。

一般而言，我们会使用安全区域组件 SafeAreaView 组件作为 ScrollView 的父组件，并给 SafeAreaView 组件设置布局属性 `flex:1`，让内容自动撑高 SafeAreaView。使用 SafeAreaView 作为最外层组件的好处是，它可以帮我们适配 iPhone 的刘海屏，节约我们的适配成本，示例代码如下：

```
<SafeAreaView style={{flex: 1}}>
  <ScrollView>
    <Text>1</Text>
```

```
<ScrollView/>
</SafeAreaView>
```

了解完 ScrollView 组件的基本使用方法后，我们再来看下 ScrollView 的性能，看看如果使用 ScrollView 来实现无限列表会怎么样。

你可以看看下面这段代码：

```
// 10 个 item 就能填满整个屏幕，渲染很快
// 1000 个 item 相当于 100+ 个屏幕的高度，渲染很慢
const NUM_ITEMS = 1000;

const makeContent = (nItems: number, styles: any) => {
  return Array(nItems)
    .fill(1)
    .map((_, i) => (
      <Pressable
        key={i}
        style={styles}>
        <Text>{'Item ' + i}</Text>
      </Pressable>
    ));
};

const App = () => {
  return (
    <SafeAreaView style={{flex: 1}}>
      <ScrollView>{makeContent(NUM_ITEMS, styles.itemWrapper)}</ScrollView>
    </SafeAreaView>
  );
};
```

上面这段代码，说的就是使用 ScrollView 组件一次性直接渲染 1000 个子视图，这里没有做任何懒加载优化。

以信息流业务为例，用户进入页面后第一眼看到的只有屏幕中的信息，一般不超过 10 条。一次性渲染 10 条信息，其实很快，就是一眨眼的功夫。但如果是 1000 条呢？算力乘以 100，内存乘以 100，耗时也乘以 100，渲染速度就慢下来了。大量的计算和内存浪费在了用户看不到的地方。

使用 ScrollView 组件时，ScrollView 的所有内容都会在首次刷新时进行渲染。内容很少的情况下当然无所谓，内容多起来了，速度也就慢下来了。

那有什么优化方案吗？你肯定想到了一些优化方案，比如按需渲染。

我参加过使用 React Native 开发的、类似抖音的视频流页面，用的就是按需渲染。用户始终只会看到当前屏幕显示的视频、下一个视频和上一个视频，我们只需要用 ScrollView 渲染 3 个视频就能满足用户的所有操作。这样做，无论用户怎么翻页，内存中就只有 3 个视频，当然也不会卡了。

刚刚说的视频流按需加载，做起来是相对容易一些的，因为只用控制 3 个视频就可以了。但类似微信朋友圈、京东首页这种一屏有多条信息内容的复杂列表页，手动按需加载就麻烦很多。那有没有“自动”的按需加载方案呢？有。

FlatList：按需渲染的列表组件

FlatList 列表组件就是“自动”按需渲染的。

FlatList 是 React Native 官方提供的第二代列表组件。FlatList 组件底层使用的是虚拟列表 VirtualizedList，VirtualizedList 底层组件使用的是 ScrollView 组件。因此 VirtualizedList 和 ScrollView 组件中的大部分属性，FlatList 组件也可以使用。关于 FlatList 更具体的使用方法，你可以查看[官方文档](#)。现在，我们还是回到 FlatList 的原理，先从理论层面上理解 FlatList 为什么可以自动按需渲染。

我们要知道，列表组件和滚动组件的关键区别是，列表组件把其内部子组件看做由一个个列表项组成的集合，每一个列表项都可以单独渲染或者卸载。而滚动组件是把其内部子组件看做一个整体，只能整体渲染。而自动按需渲染的前提就是每个列表项可以独立渲染或卸载。

简单地讲，FlatList 性能比 ScrollView 好的原因是， FlatList 列表组件利用按需渲染机制减少了首次渲染的视图，利用空视图的占位机制回收了原有视图的内存，你可以对比一下二者的区别：

```
// 从上到下滚动时的渲染方式
// SrollView 渲染方式：一次渲染所有视图
SrcollView0_9 = [{👁️},{ },{ },{ },{ }] // 浏览0~9条列表项
SrcollView10_19 = [{ },{👁️},{ },{ },{ }] // 浏览10~19条列表项
```

```

SrcollView20_29 = [{ }, { }, {👁}, { }] // 浏览20~29条列表项
SrcollView30_39 = [{ }, { }, { }, {👁}] // 浏览30~39条列表项

// FlatList 渲染方式：按需渲染，看不见的地方用 $empty 占位
FlatList0_9 = [{👁}, { }] // 浏览0~9条列表项
FlatList10_19 = [{ }, {👁}, { }] // 浏览10~19条列表项
FlatList20_29 = [$empty, { }, {👁}, { }] // 浏览20~29条列表项
FlatList30_39 = [$empty, $empty, { }, {👁}] // 浏览30~39条列表项

```

在上面的示例中，同样是渲染 40 条列表。ScrollView 一次性渲染了 40 条列表，无论你滚动到哪儿，所有的列表项都是渲染好的。

但FlatList 在你浏览 0~9 条列表项时，只渲染了0~19条列表，剩余的20~39条列表项是没有渲染的。在你浏览滚动到第 10~19 条时，FlatList 把 20~29 条列表项提前加载出来了，这就是按需渲染加载机制.当你继续滚动到 20~29 条列表项时，FlatList 会把第 0~9 条列表项回收，用空元素 \$empty 代替，当你再滚动到 30~39 条列表项时，同理 10~19 条列表项也会被空元素 \$empty，这就是内存回收。

40 条列表只是一个假设的例子，实现 FlatList自动按需渲染的思路具体可以分为三步：

1. 通过滚动事件的回调参数，计算需要按需渲染的区域；
2. 通过需要按需渲染的区域，计算需要按需渲染的列表项索引；
3. 只渲染需要按需渲染列表项，不需要渲染的列表项用空视图代替。

第一步，计算按需渲染区域。具体地说，每次你滚动页面，都会触发滚动组件 ScrollView 组件的一个“异步”回调 onScroll 事件。

在 onScroll 事件中，我们可以获取到当前滚动的偏移量 offset 等信息。以当前滚动的偏移量为基础，默认向上数 10 个屏幕的高度，向下数 10 个屏幕的高度，这一共 21 个屏幕的内容就是需要按需渲染的区域，其他区域都是无需渲染的区域。这样，即便是异步渲染，我们也不能保证所有 JavaScript 执行的渲染任务都实时地交由 UI 线程处理，立刻展示出来。但因为有这 10 个屏幕的内容作为缓冲，用户无论是向上滚动还是向下滚动，都不至于一滚动就看到白屏。

现在我们知道了按需渲染的区域，接着要计算的就是按需渲染列表项的索引。FlatList 内部实现就是通过 setState 改变按需渲染区域第一个索引和最后一个索引的值，来实现按需渲染的

怎么计算按需渲染列表项的索引呢？接着我们继续看第二步。这里我们分两种情况，第一种是列表项的高度是确定的情况，另外一种为列表项的高度是不确定的情况。

如果设计师给的列表项的高度是确定的，那么我们在写代码的时候，就可以通过获取列表项布局属性 `getItemLayout` 告诉 `FlastList`。在列表项高度确定，且知道按需渲染区域的情况下，“求按需渲染列表项的索引”就是一个简单的四则运算的问题，程序能够准确地计算出来。

如果设计师给的 UI 稿中是不定高的列表项，也就是高度是由渲染内容决定的。你没有办法在写代码的时候把列表项的高度告诉 `FlastList` 了，那么 `FlastList` 就要先把列表项渲染出来才能获取高度。对于高度未知的情况，`FlastList` 会启用列表项的布局回调函数 `onLayout`，在 `onLayout` 中会有大量的动态测量高度的计算，包括每个列表项的准确高度和整体的平均高度。

在这种列表项高度不确定，而且给定按需渲染区域的情况下，我们可以通过列表项的平均高度，把按需渲染列表项的索引大致估算出来了。即便有误差，比如预计按需渲染区域为上下 10 个屏幕，实际渲染时只有上下 7、8 个屏幕也是能接受的，大部分情况下用户是感知不到的屏幕外内容渲染的。

但是，实际生产中，如果你不填 `getItemLayout` 属性，不把列表项的高度提前告诉 `FlastList`，让 `FlastList` 通过 `onLayout` 的布局回调动态计算，用户是可以感觉到滑动变卡的。因此，如果你使用 `FlastList`，又提前知道列表项的高度，我建议你吧 `getItemLayout` 属性填上。

第三步，渲染需要按需渲染列表项。有了索引后，渲染列表项就变得很简单，用 `setState` 即可。

假设 1 个屏幕高度的内容由 10 个列表项组成。在首次渲染的时候，按需渲染的列表项索引是 0~110，这时会渲染 11 个屏幕高度的内容。当用户滑到第 11 个屏幕时，索引就是 0~210，这时再在后面渲染 10 个屏幕高度的内容。当用户滑到第 21 个屏幕时，索引是 100~310，会再在后面渲染 10 个屏幕高度的内容，同时把前面 10 个屏幕高的内容用空视图代替。当然这个过程是顺滑的，列表项是一个个渲染的，而不是 1 个屏幕或 10 个屏幕渲染的。

RecyclerView：可复用的列表组件

聊完 `FlastList`，我们再来看下 `RecyclerView`。

RecyclerView 是开源社区提供的列表组件，它的底层实现和 FlatList 一样也是 ScrollView，它也要求开发者必须将内容整体分割成一个个列表项。

在首次渲染时，RecyclerView 只会渲染首屏内容和用户即将看到的内容，所以它的首次渲染速度很快。在滚动渲染时，只会渲染屏幕内的和屏幕附近 250 像素的内容，距离屏幕太远的内容是空的。

React Native 的 RecyclerView 复用灵感来源于 Native 的可复用列表组件。

在 iOS 中，视图 UITableView，实际就是可以上下滚动、左右滚动的可复用列表组件。它可以通过复用唯一标识符 reuseIdentifier，标记视图中的复用单元 cell，实现单元 cell 的复用。

在 Android 上，动态列表 RecyclerView 在列表项视图滚出屏幕时，不会将其销毁，相反会把滚动到屏幕外的元素，复用到滚动到屏幕内的新的列表项上。这种复用方法可以显著提高性能，改善应用响应能力，并降低功耗。

如果你只开发过 Web，你可以这样理解复用：原来你要销毁一个浏览器中 DOM，再重新创建一个新的 DOM，现在你只改变了原有 DOM 的属性，并把原有的 DOM 挪到新的位置上。

RecyclerView 的复用机制是这样的，你可以把列表比作数组 list，把列表项类比成数组的元素。用户移动 ScrollView 时，相当于往数组 list 后面 push 新的元素对象，而 RecyclerView 相当于把 list 的第一项挪到了最后一项中。挪动对象位置用到的计算资源少，也不用在内存中开辟一个新的空间。而创建新的对象，占用计算资源多，同时占用新的内存空间。

简而言之，RecyclerView 在滚动时复用了列表项，而不是创建新的列表项，因此性能好。

从使用方式看底层原理

接下来，我们从 RecyclerView 使用方式的角度，进一步地剖析其底层原理。

RecyclerView 有三个必填参数：

列表数据：dataProvider(dp)；

列表项的布局方法：layoutProvider；

列表项的渲染函数：rowRenderer。

先来看**第一个必填参数列表数据 dataProvider (dp)**。为了区分列表数据 dataProvider（第一个字母小写）和列表数据类 DataProvider（第一个字母大写），后面我会用缩写 dp 来代替列表数据，其使用方法如下：

```
const listData = Array(300).fill(1).map( (_,i) => i)

const dp = new DataProvider((r1, r2) => {
  return r1 !== r2;
});

this.state = {
  dataProvider: dp.cloneWithRows(listData),
};

this.setState({
  dataProvider: dp.cloneWithRows(newListData),
})
```

在上面代码中，我们首先通过 Array(300) 创建了一个长度为 300 的数组 listData，其内容是 0~299 的数字，我们通过它来模拟 300 条信息数据。

接着，dp 是列表数据类 DataProvider new 出来的对象，它是一个存放 listData 的数据容器。它有一个必填参数，就是对比函数。在列表项复用时，对比函数会频繁地调用，因此我们只推荐对更新数据进行 r1 !== r2 的浅对比，不推荐深对比。

第三部分代码，是我们调用 dp.cloneWithRow 方法，该方法接收 listData 数组作为参数，这时我们正式把 listData 装到了 dp 容器中。其返回值 dataProvider，就是 React 的列表状态。

第四部分代码，是我们调用 setState 方法，该方法接收 dp.cloneWithRows() 的返回的 dp 对象作为参数，dp 列表数据对象更新了，整个列表也就更新了。

接下来是**第二个必填参数，列表项的布局方法 layoutProvider**。

```
const _layoutProvider = new LayoutProvider(  
  index => {  
    if (index % 3 === 0) {  
      return ViewTypes.FULL;  
    } else {  
      return ViewTypes.HALF_RIGHT;  
    }  
  },  
  (type, dimension) => {  
    switch (type) {  
      case ViewTypes.HALF_RIGHT:  
        dimension.width = width / 2;  
        dimension.height = 160;  
        break;  
      case ViewTypes.FULL:  
        dimension.width = width;  
        dimension.height = 140;  
        break;  
    }  
  },  
);
```

layoutProvider 类初始化时，有两个函数入参。第一个入参函数是通过索引 index 获取类型 type，对应的是类型可枚举。第二个入参函数是通过类型 type 和布局尺寸 dimension 获取每个类型的宽高 width 和 height，对应的是确定宽高。

用起来很简单，但这两个入参为什么要这么设计，它们有什么用？

使用列表组件 RecyclerView 有两个前提：首先是列表项的宽高必须是确定的，或者是大致确定的；第二是列表项的类型必须是可枚举的。这两个前提，都体现在了列表项的布局方法 layoutProvider 中了。

先来看第一个前提，宽高必须确定。RecyclerView 用的是 position:absolute 的绝对定位布局，所有的列表项的宽度 width、高度 height、顶部偏移量 top、左边偏移量 left 都得在布局之前计算出来。

但实际上布局方法 layoutProvider，只需要知道列表项的宽（width）、高（height）就可以了，偏移量 top、left 可以根据宽高推算出来。比如，第 N 个列表项的偏移量 top 值，实际等于前面 N - 1 个列表项的高度之和。

如果宽高不确定呢？分两种情况，一种就是不确定的，另一种是不确定但可以转换为大致确定的。对于就是不确定的情况，RecyclerView 是无解的；对于大致确定的情况，我们可以开启 forceNonDeterministicRendering 小幅修正布局位置。

比如，信息流的标题文字少的时候是一行布局，文字多的时候是两行布局，一行两行的高度偏差不大，可以在渲染后让框架帮忙进行小幅修正。通常在用户看到之前，这种小幅修正就已经完成了，用户感知不到列表的偏移。

但如果是信息流的内容高度不确定，相差百来个像素，这种大幅修正可能会让用户察觉到，不适合使用 RecyclerView 。

再来看第二个前提，类型可枚举。可枚举很好理解，两个列表项的底层 UI 视图必须一样或者大致相似，才能只改列表数据复用列表视图。如果每个列表项的 JSX 结构完全不一样，就不存在复用的可能性。一般来说，一个类型对应一个自定义组件。

理解了确定宽高和类型可枚举两个前提后，你再来看布局方法 layoutManager 需要的两个函数入参，就能清楚它的原因了。

最后是**第三个必填参数，列表项的渲染函数：rowRenderer**。

有了数据、布局，还得有组件进行承载。列表内容被分割成了一个一个的列表项，每一个列表项展示都是独立的内容信息，而可枚举的列表项组件用于承载每条信息的载体。列表项的渲染函数 rowRenderer 的作用就是根据类型和数据，返回对应的自定义列表项组件。这块逻辑比较简单，我就不做过多讲解了。

rowRenderer 的对应代码，我也放在了这里，你可以对照查看：

```
//Given type and data return the view component
_rowRenderer(type, data) {
  //You can return any view here, CellContainer has no special significance
  switch (type) {
    case ViewTypes.HALF_RIGHT:
      return (
        <CellContainer style={styles.containerGridRight}>
          <Text>Data: {data}</Text>
        </CellContainer>
      );
    case ViewTypes.FULL:
```

```
        return (
            <CellContainer style={styles.container}>
                <Text>Data: {data}</Text>
            </CellContainer>
        );
    default:
        return null;
    }
}
```

PK: ScrollView、FlatList、RecyclerView

到这里，我相信你已经对 ScrollView、FlatList 和 RecyclerView 底层原理有了一定的了解。现在，我们再横向对比一下这三个组件，帮你加深理解。

从底层原理看：

ScrollView 内容的布局方式是从上到下依次排列的，你给多少内容，ScrollView 就会渲染多少内容；

FlatList 内容的布局方式还是从上到下依次排列的，它通过更新第一个和最后一个列表项的索引控制渲染区域，默认渲染当前屏幕和上下 10 屏幕高度的内容，其他地方用空白视图进行占位；

RecyclerView 性能最好，你应该优先使用它，但使用它的前提是列表项类型可枚举且高度确定或大致确定。

理解了底层原理，FlatList 和 RecyclerView 孰强孰弱，相信你已经有了答案。

内存上，FlatList 要管理 21 个屏幕高度的内容，而 RecyclerView 只要管理大概 1 个多点屏幕高度的内容，RecyclerView 使用的内存肯定少。计算量上，FlatList 要实时地销毁新建 Native 的 UI 视图，RecyclerView 只是改变 UI 视图的内容和位置，RecyclerView 在 UI 主线程计算量肯定少。

你也可以自己实际的体验、看看性能指标或者 Debug 一下，来佐证你的结论。

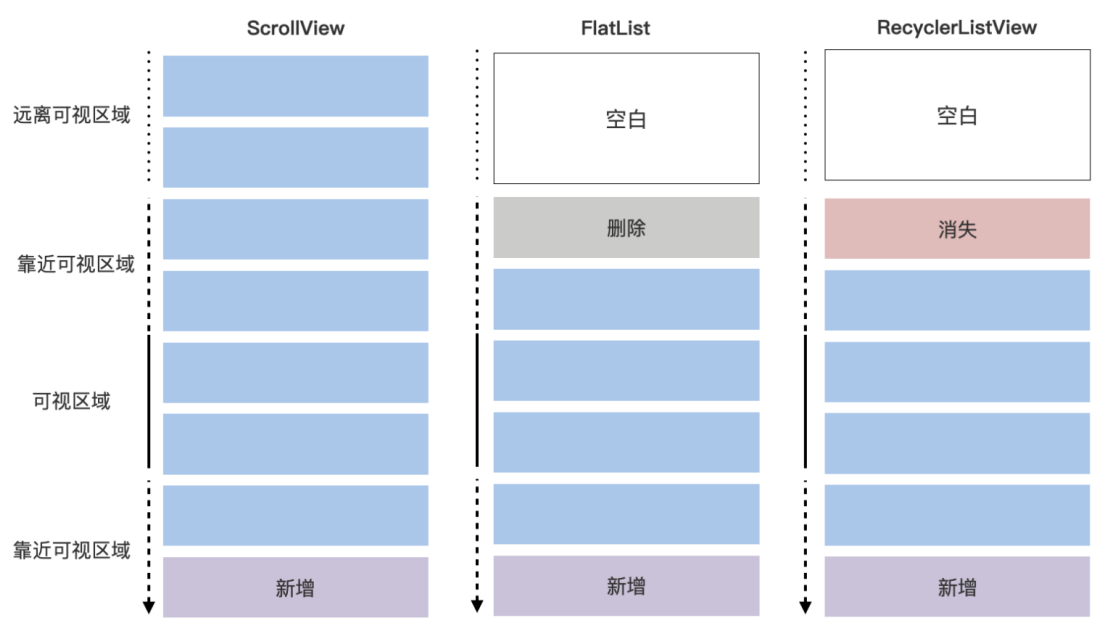
理解了底层原理，ScrollView、FlatList 和 RecyclerView 使用场景，估计你也能基本把握住了：

ScrollView 适合内容少的页面，只有几个屏幕高页面是适合的；

FlatList 性能还过得去，但我不推荐你优先使用它，只有在你的列表项内容高度不能事先确定，或者不可枚举的情况下使用它；

RecyclerView 性能最好，你应该优先使用它，但使用它的前提是可枚举且高度确定或大致确定。

这里我也总结成了两张图表，你可以看看：



	ScrollView	FlatList	RecyclerView
渲染原理	从上到下 一次渲染	从上到下 按需渲染 空白占位	绝对定位 按需渲染 复用同类组件
渲染区域	所有	当前屏幕 + 上下 10 屏幕	当前屏幕 + 上下 250 像素
滚动性能	视情况而定	iOS 优秀 安卓低端机能用	iOS 优秀 安卓低端机良好
使用场景	内容少的页面 或手动优化场景	高度不确定场景	高度确定 或大致确定场景

总结

最后，我们总结一下今天这节课所讲的重点：

1. 滚动组件 ScrollView 是列表组件 FlastList 和 RecyclerView 的底层实现，ScrollView 的绝大部分属性在 FlastList 和 RecyclerView 上都有；
2. 从按需渲染的可视区域的大小和对底层 UI 视图的操作方式上分析，RecyclerView 比 FlastList 的内存更少，在 UI 线程的计算量也更少；
3. 为了让你的无限列表性能更好，我推荐你优先使用 RecyclerView，然后才是 FlastList。

列表是一个很大的话题，牵涉到的性能优化细节和实践内容很多，这一讲可以算作列表的一个入门。

受限于手机性能，无限列表是经常出现性能问题的重灾区，我也参与和优化过一些非常复杂的无限列表场景，包括 Hybrid、小程序 和 React Native，有过非常多的实践。在后面的篇章中，我会基于这些入门知识，和你讲讲具体业务中的实践操作，还有一些新架构中无限列表的变化。

在这一讲中，我希望你能把基础打好，自己动手实践一下 ScrollView、FlastList 和 RecyclerView。同样，今天我也给你留了补充材料和作业。

补充材料

使用文档：

[ScrollView](#) 和 [FlatList](#) 你可以参考官方文档，进一步学习它们的具体使用。

RecyclerView 你可以在 Github 上找到它的[文档](#)，在作者的博客 [《RecyclerView: High performance ListView for React Native and Web》](#) 了解它的诞生背景。

实战指南：

RecyclerView 的内部状态是 renderStack 用于确定哪些视图应该渲染，[它的复用机制是通过列表项的类型 type 找到要被回收列表项 renderStack\[key\]，然后用新列表项索引 index 替换被回收的列表项索引 oldIndex。](#)

RecyclerView 是可以实现高度不确定的无限列表的。图片的高度可以通过服务端事先传过来，文字的高度可以按照我在 [《React Native 无限列表的优化与实践》](#) 一文中提供的

算法提前算出来，再开启高度动态修正。

RecyclerView 也是可以实现瀑布流布局的。RecyclerView 其实就是绝对定位 (x,y,width,height)，但不支持双列，你可以用 [patch-package](#)，把底层计算 layout 用的 “[relayoutFromIndex](#)” 和 “[this._layouts](#)” 改了。

它们的 Demo 我放在了 [GitHub](#) 上，你可以动手把玩一下。

作业

1. 请你使用 React Hook 的语法实现一个 RecyclerView 无限列表。
2. 你遇到过那些列表性能问题又是怎么解决的，能不能和我们分享一下你的心得？

欢迎在留言区分享你的见解。我是蒋宏伟，咱们下节课见。

© 版权归极客邦科技所有，未经许可不得传播售卖。页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

上一篇 07 | TextInput：如何实现一个体验好的输入框？

下一篇 09 | Fast Refresh：提高 UI 调试效率神器

精选留言 12